Foundation for Virtual and Augmented Reality Systems

Prof. Samit Bhattacharya

Department of Computer Science and Engineering

Indian Institute of Technology, Guwahati

Week – 12

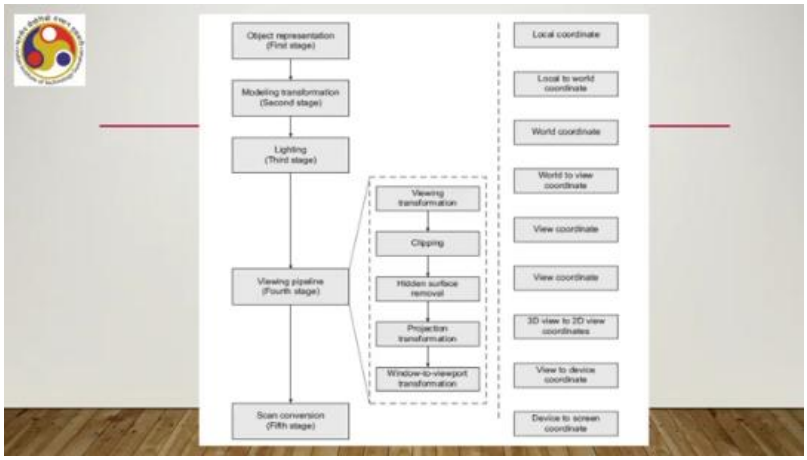
Lecture – 57

VR resource optimization

Hello and welcome to the NPTEL MOOC's course on Foundations for Virtual and Augmented Reality Systems. We will be continuing our discussion on resource optimization. In particular, we are talking about resource optimization from the point of view of rendering, which is a very important component in any extended reality applications including virtual reality and augmented reality applications. So, we will primarily focus on immersive experience, as we have already stated earlier, and we are going to relate the discussion to VR resource optimization in particular.

Although such concerns equally apply to AR, However, the emphasis is more on resource optimization in the context of VR because realistic scene generation is more important there and that requires more resources as compared to AR experiences. In order to understand resource optimization concerns, we started our discussion on computer graphics, which deals with the rendering process, rendering on a computer screen or any display device such as HMDs.

So, we learned about the basic idea of computer graphics, including the 3D graphics pipeline stages. Also, so far we have covered three of those stages, namely the first stage, object representation; the second stage, modeling transformation; and the third stage, lighting calculation or calculation of intensities at the pixel point pixel positions. In this part of the lecture, we are going to talk about the fourth stage of the pipeline, that is, view transformation. So, just to put things into perspective, we have this 3D graphics pipeline.



Now, here we have object representation, the first stage, which takes place in the local coordinate reference frame. We already covered it. Then we have modeling transformation, the second stage, also known as geometric transformation, which transfers the object definition from local coordinates to the definition in a world coordinate scene. Then we have the third stage, that is lighting calculation, which as per our pipeline discussion, takes place in the world coordinate scene. Now, this lighting calculation is quite complex, and we talked about one simple lighting model. In this part, today's part of the lecture, we are going to cover the fourth stage, that is the viewing pipeline, comprising all these transformations and other processes. And later on, we will cover the scan conversion. So, let us get the idea of the fourth stage, that is view transformation.

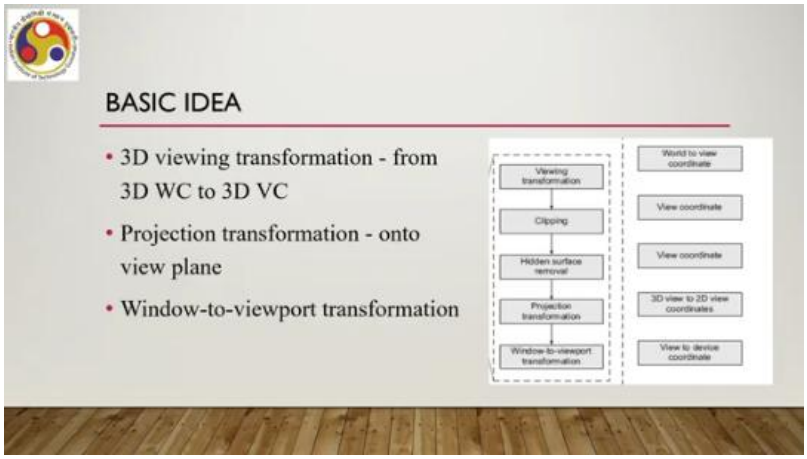
Here again, our emphasis will be on giving you the basic idea rather than going into the intricate details. So, we will try to confine our discussion to a bare minimum level. So, what is the basic idea? The idea is that when we see an image on a screen, any screen including HMD, we are basically seeing a projection of a portion of the 3D scene. So, we have a 3D scene which we have constructed and illuminated with our earlier stages.

Now, when we say that we are trying to render it on a computer screen, then essentially what we are doing is Selecting a portion of it and then projecting it from a 3D scene to a 2D display plane or 2D view plane. So, this is similar to taking a photograph. So, if you are a photographer or if you know something about capturing photos, what do we do? We typically have a camera.

Any camera, even your smartphone camera. So, when we try to capture a photo, first we look at the scene with our naked eye; that is the 3D scene. Then we look at the same scene using the camera screen. Then we capture a portion of the scene. So, the scene, our field of view covers a wide range of a wide portion of the scene. However, we do not capture the entire thing; we capture only the portion which is within the field of view of the camera. And we set the camera in such a way that our area of interest is captured. Now, when we say captured, that means we are able to see it on our screen. Now, the screen is a 2D plane, whereas the scene, the portion of the scene that we have captured, is in the 3D space.

So, then there is a transformation, a projection transformation, which transforms from 3D space to a 2D screen. The same thing we try to simulate in computer graphics with a set of stages, which are the viewing pipeline stages. So, view transformation essentially is this

viewing pipeline. It has its own sub-stages that simulate the process of capturing a photo. So first, we do a viewing transformation, a 3D viewing transformation, where the 3D world coordinate description of the scene gets transformed to a 3D view coordinate description.



So we define a view coordinate system. So, this is a 3D view coordinate. So, we define a view coordinate system, then perform a transformation operation that transforms the 3D world coordinates into the view coordinate system. Now, this is akin to saying that we are looking at the scene through the camera lens. When we capture a photo, before that, we see the scene using the camera mechanism, the camera viewing mechanism provided.

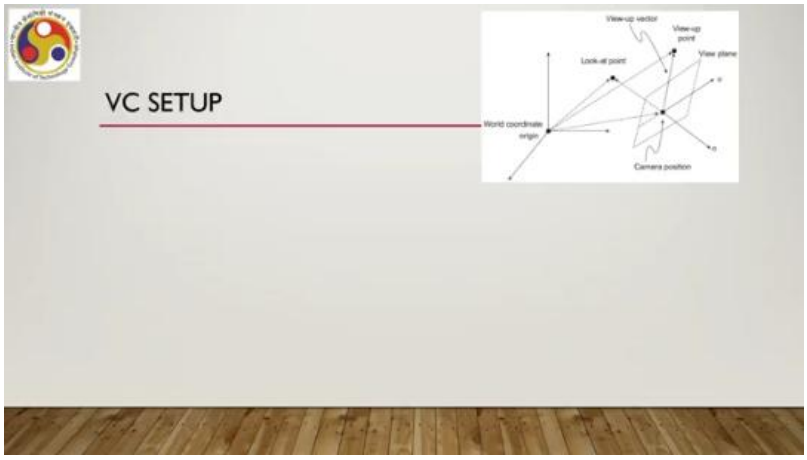
So, looking at the scene through the camera viewing mechanism and looking at the scene through our naked eyes, these two are different things. And when we are looking at the scene through the camera viewing mechanism, essentially the scene gets transformed from the world coordinate system to a view coordinate system that is defined by the camera or, in other words, the camera parameters. So, that is what happens here. The second thing is a projection transformation that projects the 3D view coordinate scene to a view plane, which is a 2D plane.

So, this is the projection transformation from the 3D view coordinate system to the 2D view coordinate system or 2D view plane. And then finally, there is another transformation, which is the window-to-viewport transformation, that happens here. Here, we transform from a 2D view plane description of the scene, or the projected scene, to a 2D device coordinate space. Now, here, there are two more operations that take place: one is clipping, and one is hidden surface removal. Clipping takes place in the 3D view coordinate system, as per our way of understanding the pipeline, and hidden surface removal again takes place in the 3D view coordinate system.

Just to emphasize here is that the way we are explaining the pipeline, it is not necessary that this sequence should be followed in implementation. So, different implementations can alter the sequence of the stages or sub-stages. So, this sequence of stages or sub-stages is not sacrosanct; it may be changed depending on the implementation. So, we have three transformations: viewing transformation, projection transformation, and window-to-viewport transformation; and two operations: clipping and hidden surface removal. Together, they constitute the viewing pipeline.

That is the fourth stage of the 3D graphics pipeline. Now, let us try to understand these transformations in brief. So, let us start with the first one, which is the 3D viewing transformation, where we transform from a 3D world coordinate description to a 3D view coordinate description. This is to simulate the camera photo-taking analogy, looking at the same scene, not with our bare eyes, but with the camera viewing mechanism.

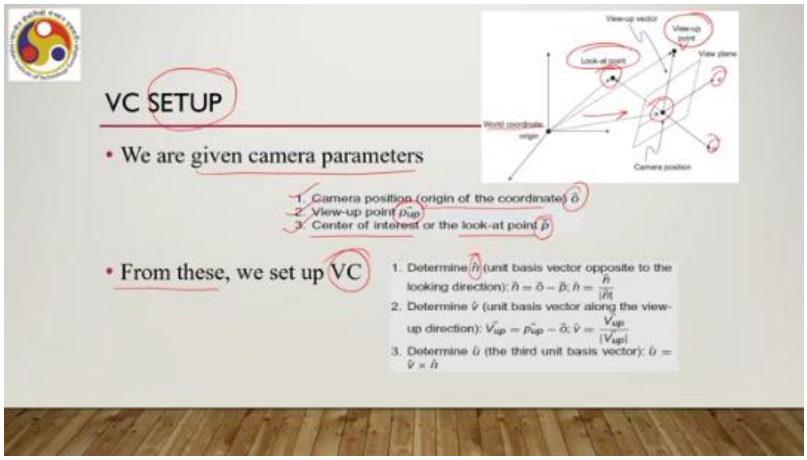
So, first, we need to define the view coordinate system, and then we perform the transformation. And this definition of the view coordinate system depends on the parameters, the camera parameters. So, the setup is something like this. So, this is the point in the world coordinate scene. This is the world coordinate system reference frame within which all the scene points are defined.



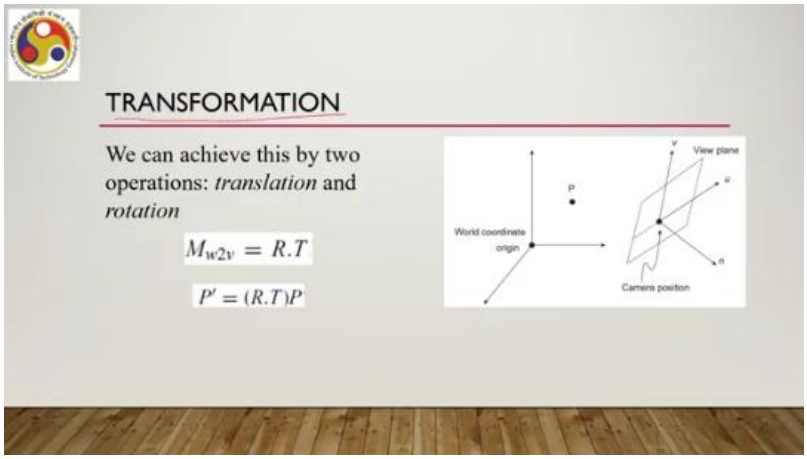
Now, we have to define a view coordinate system having its own axes u, n, and v. So, essentially what we are trying to do is transfer this point in the world coordinate to this view coordinate scene, the 3D view coordinate scene. So, we are given the camera parameters. What are those parameters? The camera position serves as the origin, which can be represented by this origin vector with respect to the world coordinate scene.

So, this is the vector. Then, Second is the view of point, again, can be another vector, the view of point, and the view direction we can define with respect to the vector. And third is the center of interest or the look-at point P. This is the center of interest or the look-at point. So, before taking a photo, we typically focus on a look-at point. or center of interest, and then capture the photo.

So, that idea is taken here. Now, from these parameters, we can set up the view coordinate system. We will skip the details. For those details, you can refer to the material cited at the end of the lecture. So, we can simply say that to determine the parameters, to determine the view coordinate system, we need to determine the three orthogonal axes N, V, and U. N can be obtained; N is the unit basis vector opposite to the looking direction and can be obtained with respect to the origin vector and the look-up direction, that is the N vector, and then the unit vector we can obtain by its value. Then, that is the first axis N. Then, the second one we can determine is the V unit basis vector along the Vup direction, that is, we can first get the Vup direction, that is Vup - O, and then we get the unit basis vector as this ratio. Once these two, n and v, are obtained, remember n is obtained using the origin vector and the look-at point, v is obtained using the view up point and



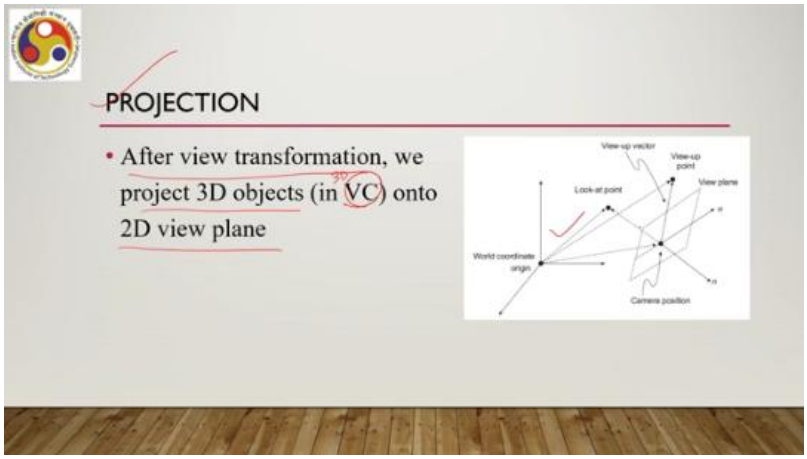
the origin point—and once these two are determined, we can determine u, the third axis and the third unit basis vector, as u simply is the cross product of v and n. So, that is how we can determine the three mutually orthogonal axes, and these three axes together define the view coordinate system. Now, once we have the view coordinate system, that is the three axes, we can transform the world coordinate description to the view coordinate description by two basic transformation operations: translation and rotation.



So, the composite transformation matrix is the dot product of these two transformation matrices. r and t, and then any point in the view coordinate system is obtained from the corresponding point in the void coordinate system after the point vector is multiplied with this composite transformation matrix comprising r and t. So, if a point P is here and if we want to transform it to a point in the view coordinate system, then we can do so using this formulation. Again here, how to, what is the matrix for M that is the composite matrix, or rather what are the individual matrices for R and T? These details we are skipping.

You can refer to the material cited at the end for those details. So, that is about view transformation. So, we define a view coordinate system with respect to three camera parameters, and then we use that information to transform a point from the void coordinate system to the view coordinate system, and those matrices are formed on the basis of those axis vectors and their scalar components.

Next is projection transformation. After view transformation, we project 3D objects in the view coordinate system, the 3D view coordinate system, onto a 2D view plane. So, this is our world coordinate description; it gets transformed to a 2D view plane description. So, in between, we first transform it to a 3D view coordinate description, and from there we project it to a 2D view plane description.

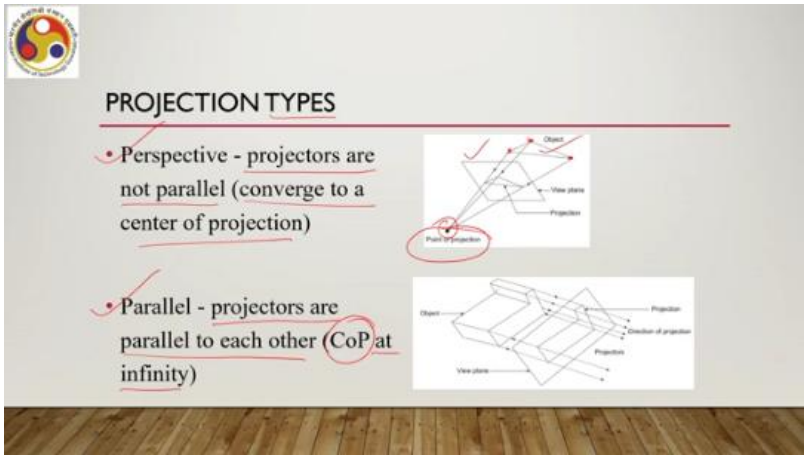


So, this is the view plane. So, such view planes are usually defined as a rectangular area. which is often called a clipping window. So, there are different terminologies used to refer to more or less the same concept. Now, in order to perform projection, apart from the clipping window, we also define a 3D volume, a 3D region in the scene that is basically what portion of the scene we are going to project. This is known as the view volume. So, the view volume defines the portion of the scene that we are going to project on the view plane. Objects that are inside this view volume are projected on the view plane, and objects that are outside are discarded. Now, when we say that we are discarding objects that are outside of this view volume but part of the scene, we call that a clipping process. So, clipping takes place, as per our description of the pipeline, before we perform the projection. Now, when we say we want to perform projection, projection can be of different types, and we also have to determine the computation based on the type of projection that we want to have. So, one type is perspective projection. Here, the projectors, which are lines emanating from the object in the 3D view coordinate system and intersecting the view plane, these projectors are not parallel to each other; they tend to converge to a center of projection. Something like this. So, this is the object, suppose. Now, if we extend some lines from these vertices of the object, and this is the view plane. So, they intersect the view plane; the points where they intersect define the projection of that 3D object on the 2D view plane.

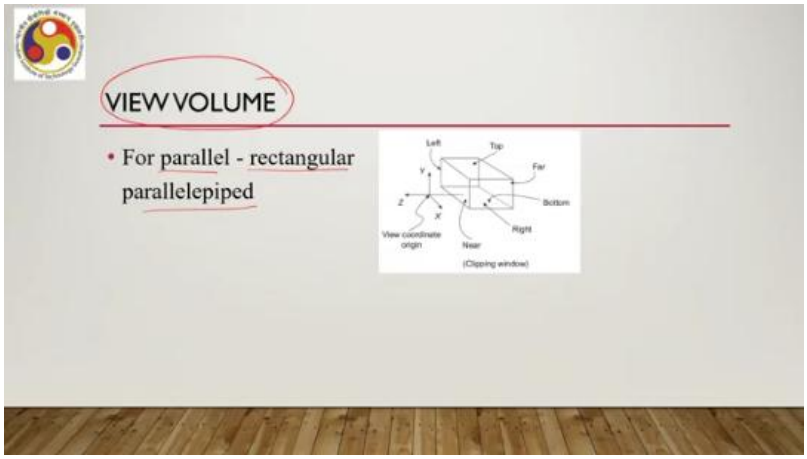
In the case of perspective projection, those lines tend to meet at a distance or converge at a distance. That is called the center of projection or point of projection. So, this perspective projection gives us realistic rendering, a realistic feeling. So, if we want to generate a realistic scene, then we should stick to perspective projection.

Now, there is another type of projection that is called parallel projection. Here, the projectors are parallel to each other; that is, the center of projection or the point of projection is at infinity, which means they never meet. This is something like that, as

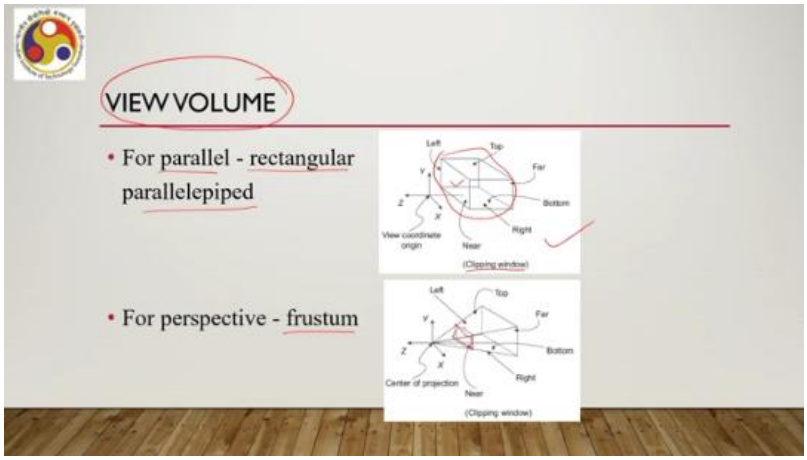
illustrated here. The idea is something as illustrated in this figure, where we have an object; projectors are shown here. They intersect the view plane and give us this projection.



However, the projectors are parallel to each other, so they do not meet. And we say that in such cases, we call that parallel projection, where we say that the projectors meet at infinity. So, for parallel projection, the view volume, if we want to generate an effect resulting from parallel projection, then we wish to have a view volume that is a rectangular parallelepiped-shaped region as shown here. And this volume defines the objects that are to be projected on the clipping window.

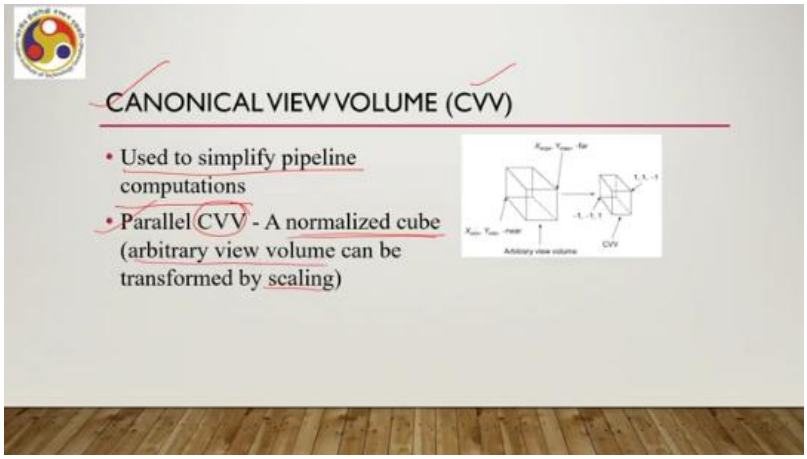


So, typically such volumes have one face as the clipping window, as shown here, and then we have other faces. For perspective projection, such a view volume takes the shape of a frustum, as shown here. Again, one side of the frustum is the clipping window. Our objective is that whatever objects are within this frustum are to be projected on this clipping window. Now, there is a concept called a canonical view volume.



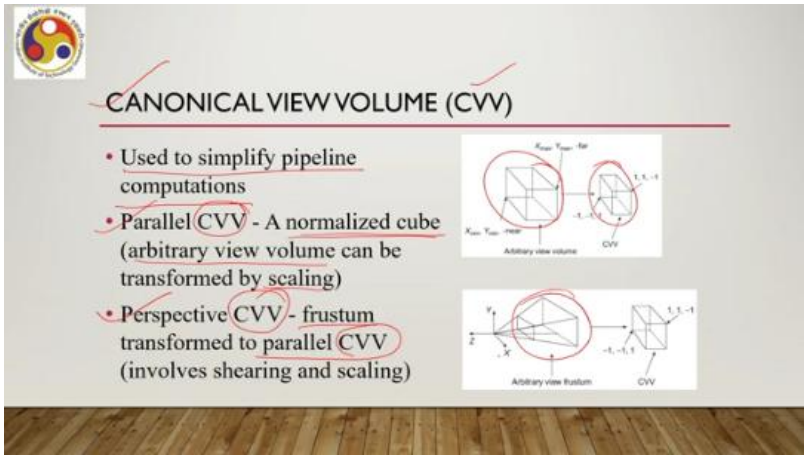
So, remember we talked about the clipping operation. That is, discarding objects that are outside the view volume. Now, to discard the objects, if we go by the naive method, the simplest most intuitive method, then we have to compute the intersection of the view volume boundaries with the object boundaries. Now, if we are dealing with arbitrarily shaped view volumes such as the frustums, then typically calculation of those surfaces and of view volume boundary intersection points turns out to be quite computation intensive.

In order to simplify that, we take recourse to a concept called canonical view volume, which is used to simplify the pipeline computations. The idea is that we use a normalized cube to define the view volume. That means whatever view volume is defined, whether for parallel or perspective projection of arbitrary shape and size, we convert those to a normalized cube. Now, for parallel projection, such a canonical view volume can be obtained by scaling the arbitrarily defined view volume for the parallel projection. The idea is shown here. This is a view volume defined for parallel projection arbitrarily, and this is a transformed view volume which is called the canonical view volume for parallel projection, which is a unit cube. For perspective projection, the computation is a little bit higher.

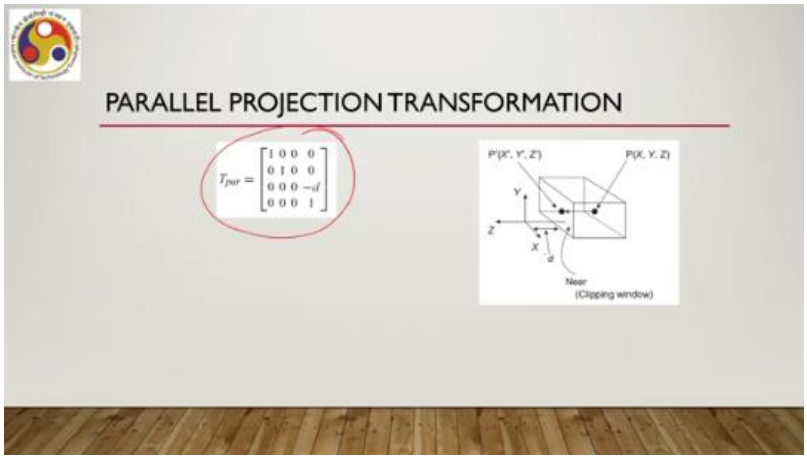


So, here we need to convert the frustum to a parallel canonical view volume, that is, a normalized unit cube. So, to convert a frustum to a normalized unit cube such as this requires scaling as well as shearing operations to change the shape along with the change

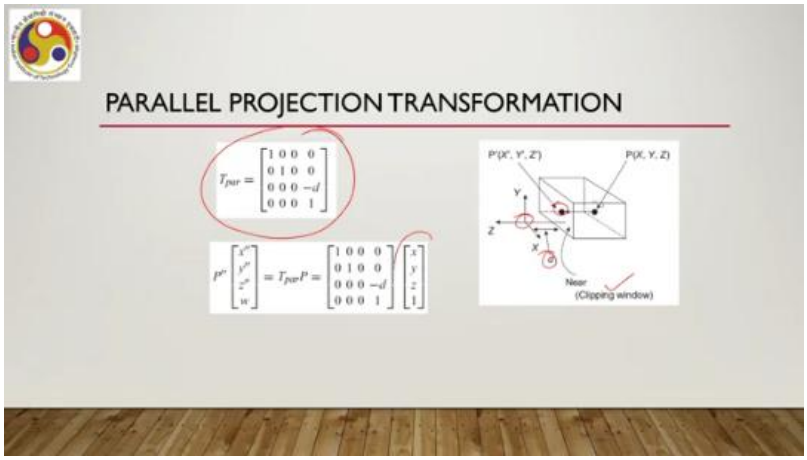
in size. So, here the amount of computation involved in such a transformation is a bit more as compared to parallel projection. Now, let us quickly talk about the two projections: how to perform the projections.



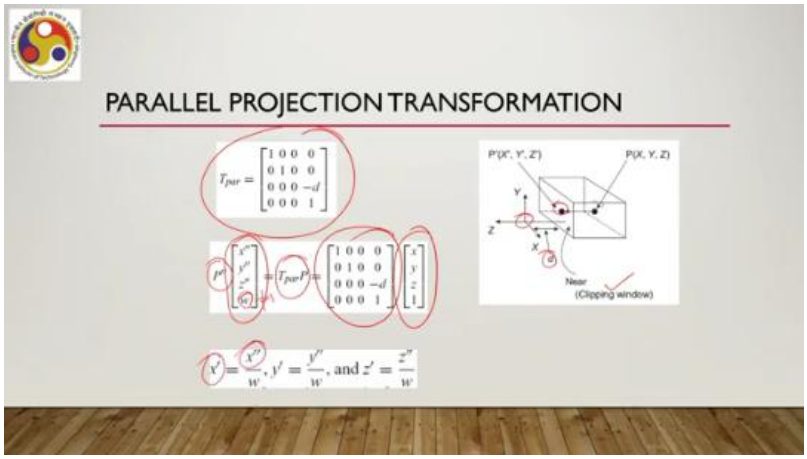
Like before, here also we take recourse to matrices projection transformation matrices. So, we multiply the point vectors with the transformation matrix to get the requisite projection. So, for parallel projection we have a matrix like this. This is a very simplified representation of the matrix where D is the distance of the clipping window from the origin of the 3D view coordinate system.



So, then we have such a representation in a 4 by 4 matrix and to get a projected point on this clipping window, what we need to do is, we need to have this point vector defined in the 3D view coordinate system, we multiply it with the projection matrix to get this transformed point in the form of a point vector. However, here we have this homogeneous factor w which is typically not equal to 1.

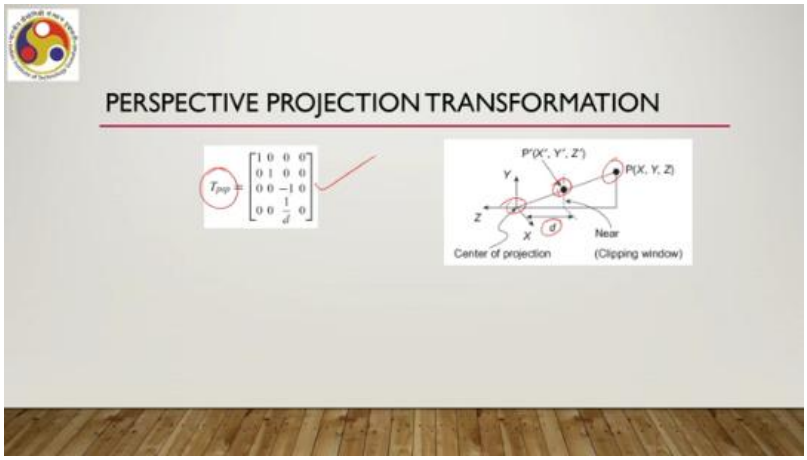


So, to get to the Cartesian points what we need to do is basically to divide the obtained coordinate values by this homogeneous factor. So, to get Cartesian value along the x direction, we need to divide this corresponding vector value by the homogeneous coordinate, similarly for y and z. We need to do this additional computation here. What about perspective projection transformation? So, we have a point here, we want to project it on the clipping window here.

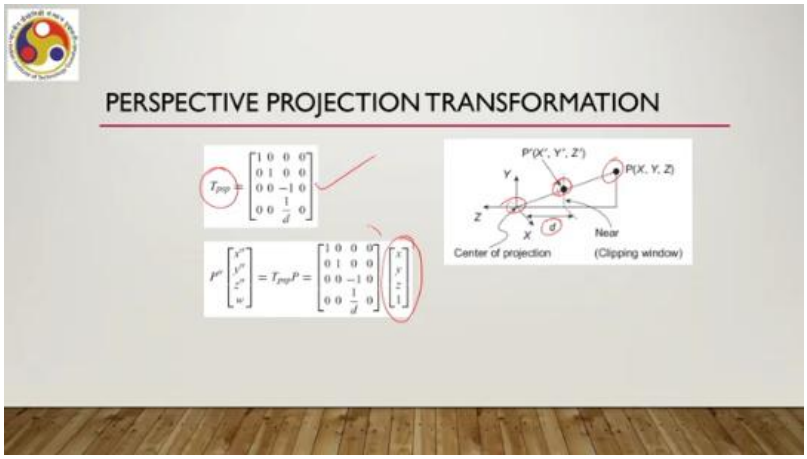


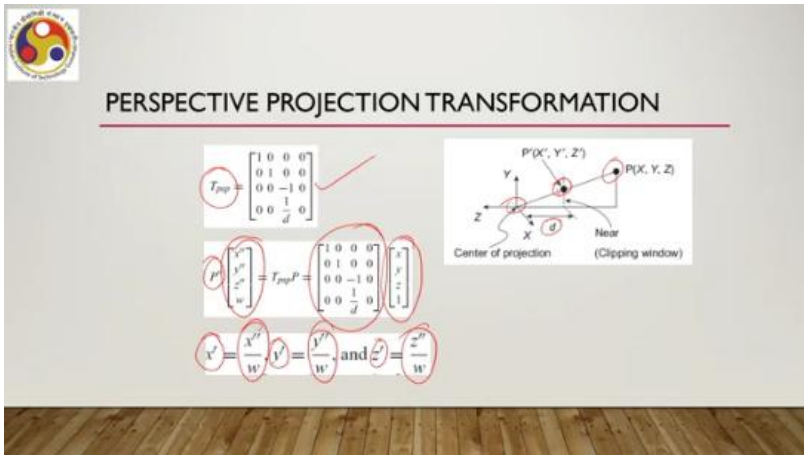
and D is, as usual, the distance of the window from the view coordinate origin. Here, D is the distance of the window from the center of projection. Now, the center of projection may typically be assumed to be at the origin. So, accordingly, the setup can be made. However, that is not necessarily the case always.

But let us assume that D is the distance from the clipping window to the center of projection. So, in that case, we have a perspective projection matrix in this form shown here, which is a bit more complex as compared to the parallel projection matrix. And then we follow the same procedure. We have this point vector in the 3D view coordinate system.



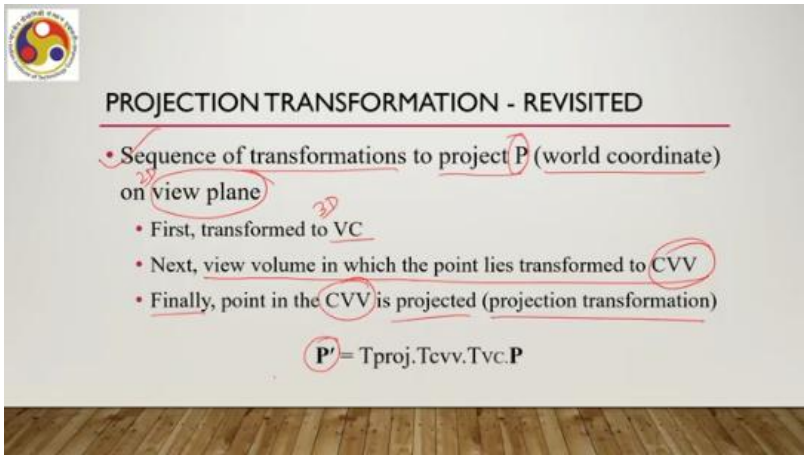
We multiply it with the projection matrix, here the perspective projection matrix, to get the projected point in the homogeneous coordinate form, and then to get to the Cartesian coordinate form we need to divide by the homogeneous factor, as we have done before for parallel projection. This is for perspective projection. So, here again, we are skipping many different types of projections under each of these categories, that is, parallel and perspective. Here, we are just trying to give you a very broad idea of projection and how it can be done.

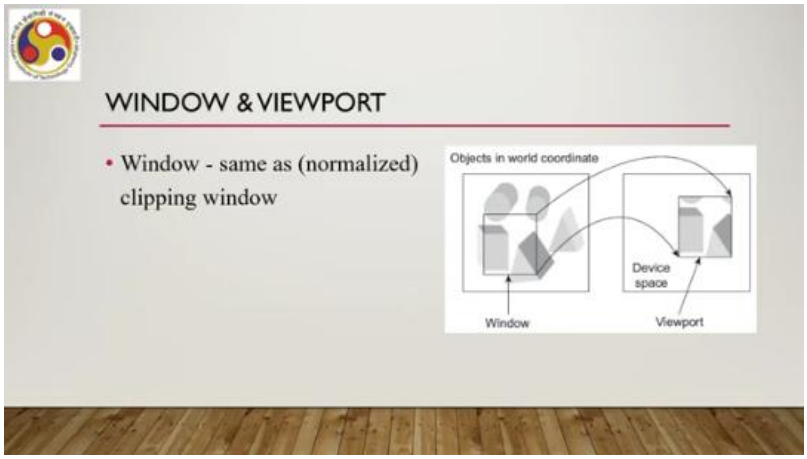




So, then, let us try to enumerate the sequence of transformations that are involved to project a point P in the world coordinate system to a point on the 2D view plane. So, first it gets transformed to the view coordinate system, the 3D view coordinate system. Then the view volume in which the point lies is transformed to a canonical view volume, which is typically the case before we perform the clipping operation. And finally, the point in the canonical view volume is projected onto the view plane. through the projection transformation.

So, if P' is the final point that we obtain on the view plane, the 2D view plane, then we can compute P' in terms of the original point P in the world coordinate scene and multiplying P with the transformation matrices for view coordinate, world coordinate to 3D view coordinate transformation matrix, then a matrix to transform to the canonical view volume, and finally the projection transformation matrix. So, these are the matrices we need to multiply together to get a composite matrix representing the overall projection from the world coordinate scene to the 2D view plane. That is up to projection to the view plane.

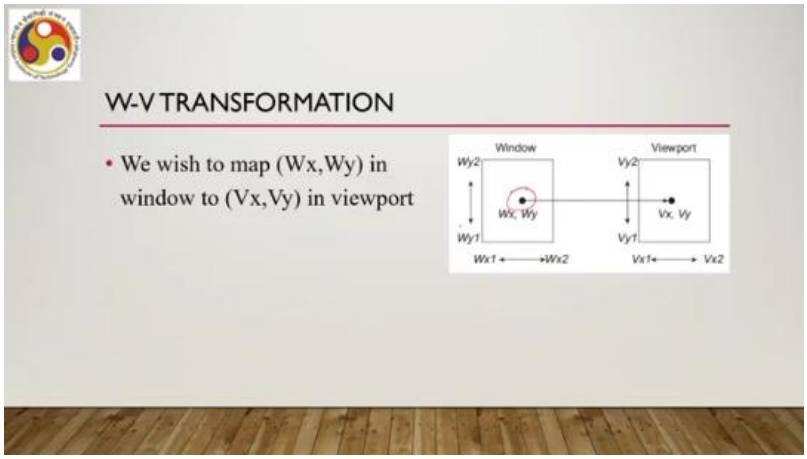




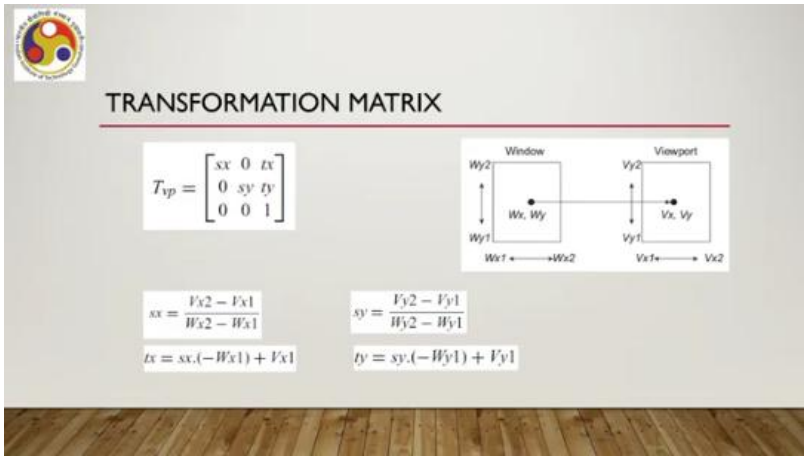
Now, there is one more projection, which is window to viewport. Now, what is a window? A window is the same as a normalized clipping window. So, typically there is another term for this view plane, which is window. This is actually a normalized clipping window.

The viewport, on the other hand, is not defined on the view plane; it is defined in the device space, that is, a portion of the final screen where the scene will be rendered. So, this is defined with respect to the screen origin and the screen dimensions or display origin and display dimensions. The viewport has nothing to do with the display; it is an abstract intermediate representation. The window has nothing to do with the display; it is an abstract intermediate representation. The viewport is defined with respect to the display origin or screen origin and screen dimensions.

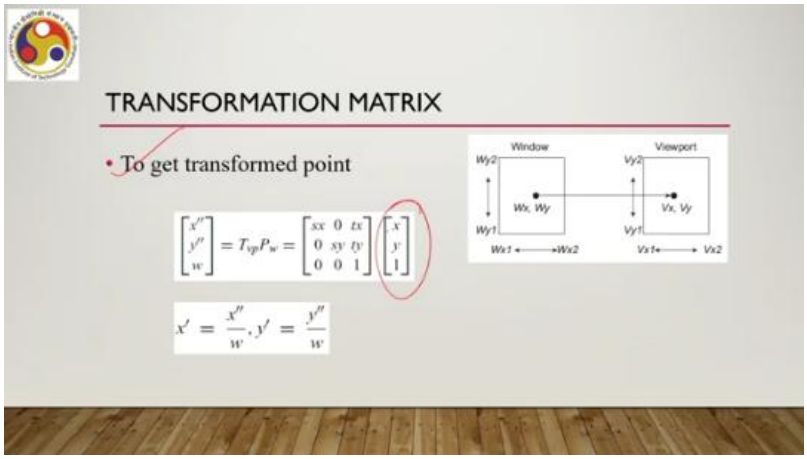
So, we need to map from this abstract representation in the window to a device-specific representation in the viewport. In other words, given a point on the window, we need to transform it to a point on the viewport. So, if a point is represented in the form Wx, Wy on the window, we want to map it to a point Vx, Vy on the viewport. So, here I will just talk about the transformation matrix. I will not dwell on how these matrices or how this matrix is particularly obtained.



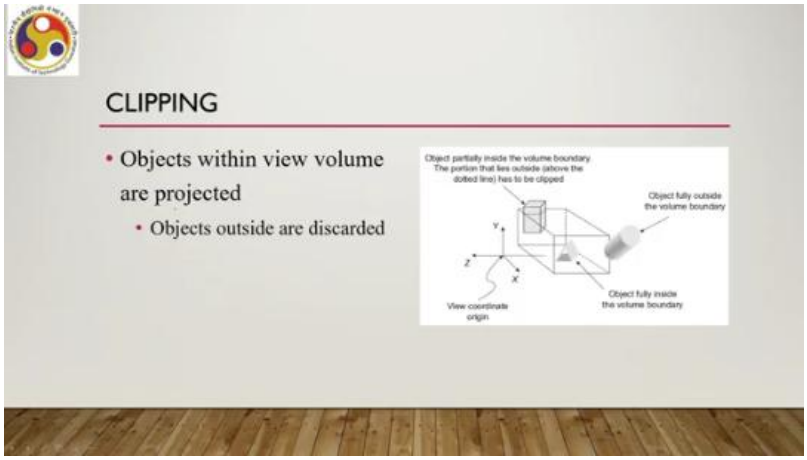
That can be obtained by simple geometric operations and correspondence between the two representations. For the derivation again, I would like to request you to refer to the material cited at the end. So, the transformation matrix can be represented in the form of this 3 by 3 matrix, where Sx is defined in terms of the viewport dimensions and window dimensions. Sy is similarly defined with respect to the viewport dimensions and window dimensions along the y-axis; Sx is the dimensions along the x-axis. Then Tx can be computed in terms of Sx and the window dimension and viewport dimension along x.



Ty can be computed in terms of Sy and again the window and viewport dimensions along y. So, once these are computed, then we can have this transformation matrix Tvp, and to get to the point, to get the transformed point, what we need to do is basically multiply the point with this transformation matrix. So, we have this point vector transformation matrix; after multiplication, it will yield the transformed point. So, to get the transformed point, we have this window point Wx, Wy.



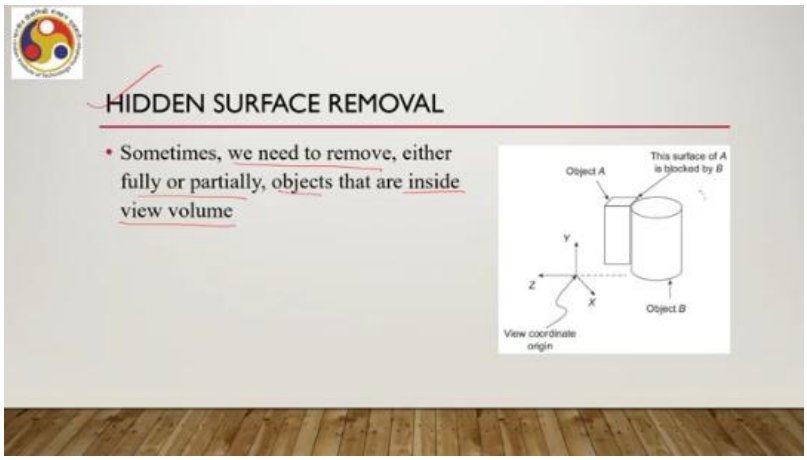
We multiply it with the transformation matrix to get the transformed point as shown here, and to get to the Cartesian points, what we need to do is divide the obtained point with this homogeneous factor W. So, here the representation is again homogeneous coordinate representation, and to get back to the Cartesian point, we need to divide the obtained point with the homogeneous factor W. Here we should note that there are two more processes that take place in this stage, which we have mentioned earlier: one is clipping, one is hidden surface removal. So, in clipping, what happens is that we define a view volume; the objects that lie outside the view volume are removed or eliminated. Only the objects that are inside are retained.



So, it is done before projection because then the projection operations need to be applied on a fewer number of vertices, and the corresponding computational complexity will be diminished or reduced. So, the idea is illustrated in this figure here. As you can see, suppose this object; we have this view volume. This portion of the object is outside. So, we have to determine this portion of the object and remove it.

In this case, the whole object is outside. So, we just discard the whole object, whereas in this case the whole object is inside, so we need to retain the whole object. So, this process of discarding objects, that is, identifying which part to keep and which part to discard, is the clipping process. Now, another operation that takes place in this stage is hidden surface removal. Although with clipping we are removing objects that lie outside,

sometimes even within the view volume, it is possible that with respect to the viewer position, some portion of the objects or some objects altogether are not visible. So, in such cases, we need to remove either fully or partially objects that are inside the view volume, but not visible or not supposed to be visible with respect to a particular viewer position or view position. The idea is illustrated here. So, both these objects are within the view volume, and we ideally need to project them.



However, if the viewer is looking at these two objects from this direction, then you can see that this surface will be occluded by this front object with respect to the viewer or viewing

direction. So, then projecting this surface makes no sense. So, to further reduce computational complexity, we ideally should remove this surface before projecting the other vertices onto the view plane. So, this process of identifying objects that are not visible to a viewer or from a particular viewing position is called hidden surface removal.

So, object A is partially blocked from the viewer by B, and if the viewer position changes to this direction, then object B will be partially blocked by object A. So, for realistic image generation as well as to reduce computation, the blocked portion of A should be eliminated before the scene is rendered. And in order to do that, we need to remove these surfaces or these surface points from further processing in the pipeline stages. That is known as hidden surface removal. That type of removal of surfaces or surface points that ideally should not be rendered with respect to the viewing position is called hidden surface removal.

There are a number of algorithms for each of these operations, namely clipping and hidden surface removal. They have their own complexities and, of course, introduce more and more computations into the overall process. However, as is the case with our previous topics, we will skip those here for simplicity. Our objective is just to give you some idea of the amount of computation involved, not to get into the details of the actual computations. But for those of you who might be interested in learning more,

please refer to the material cited at the end for more such details. So, let us summarize what we have learned so far. So, in this part of the lecture, we covered one more stage of the graphics pipeline, that is the fourth stage, which is the viewing transformation. Essentially, it is a pipeline in itself involving multiple transformations: view transformation, which transforms a world coordinate scene to a 3D view coordinate scene; projection transformation, which transforms a 3D view coordinate scene to a 2D view plane description; and finally,

viewport transformation, which transforms a 2D view plane description to a 2D device space description. Also, we mentioned two operations: clipping and hidden surface removal. What are those operations, and what needs to be done to carry out those operations? In the next part of the lecture, we will cover the remaining stage of the pipeline, which is the fifth stage: scan conversion. Along with that, we will have some more discussion on the implications of the knowledge that we have gained so far.

So, I hope that you could follow the topic and understand the topic that we discussed. Of course, we have skipped a lot of details. Those details are not relevant for the present course of discussion. However, for those of you who are more interested, please refer to the material cited at the end of this whole discussion. So, I look forward to meeting you in the next part of the lecture.

Until then, thank you, goodbye, and happy learning.